

Power-Aware Scan-Based LBIST for a MIPS32 Processor Using Dynamic Toggle-Rate Control

Siddhant Shah
(B23334)

Akash Kumar
(B23481)

Somya Bhadada
(B23052)

Contents

1	Introduction	3
1.1	Motivation and Project Overview	3
1.2	Main Components Used	4
2	Overall LBIST Architecture	5
3	Module Descriptions	6
3.1	32-bit LFSR-Based Test Pattern Generator	6
3.2	Phase Shifter (PSF)	6
3.3	Programmable Low-Pass Filter (PLPF)	7
3.3.1	Static PLPF Modes ($n = 1, 2, 3$)	7
3.3.2	Dynamic PLPF with (α, β, γ)	7
3.3.3	Seed Dependence	8
3.4	Scan Chain Design and Operation	9
3.5	Single-Cycle MIPS Architecture and Modifications	9
3.6	MISR and Output Response Analysis	11
4	Results and Waveforms	11
5	Conclusion	12

Abstract

This project implements a complete Logic Built-In Self-Test (LBIST) infrastructure around a 32-bit single-cycle MIPS processor in VHDL. The goal is to generate pseudo-random test patterns on-chip, apply them through a scan chain, and compact the responses into a Multiple Input Signature Register (MISR), while explicitly controlling switching activity during test using a Probabilistic Switching Factor (PSF) and a Programmable Low-Pass Filter (PLPF).

The test pattern generator is based on a 32-bit Linear Feedback Shift Register (LFSR) with polynomial $x^{32} + x^{22} + x^2 + x + 1$. A phase shifter (PSF) and a dynamic PLPF are used in series to shape the statistics of the scan-in stream. The scan chain is wrapped around a single 32-bit MIPS pipeline stage, allowing instructions and ALU results to be driven and observed through scan. A MISR with the same polynomial as the LFSR compresses the scan-out stream into a 32-bit signature that is checked against a golden reference in the testbench.

The report presents the architecture, detailed description of each module (LFSR, PSF, PLPF, scan chain, MIPS core and MISR), and quantitative results: measured toggle rates on the scan-in signal for various PLPF configurations (α, β, γ) and several LFSR seeds, together with the corresponding MISR signatures.

1 Introduction

1.1 Motivation and Project Overview

As integrated circuits grow in complexity, external tester access to internal nodes becomes increasingly limited. Logic Built-In Self-Test (LBIST) is a widely used design-for-testability (DFT) technique that allows a digital system to test itself using on-chip pattern generation and response compaction hardware. LBIST is attractive because it:

- Reduces test time and tester memory requirements.
- Enables in-field and at-speed self-test.
- Scales more easily than external scan-based testing.

However, pseudo-random LBIST patterns typically have a switching activity close to 50 %, which can lead to excessive dynamic power, IR-drop and potential reliability issues during test. This project addresses that problem for a 32-bit single-cycle MIPS processor by:

- Generating pseudo-random patterns using an on-chip LFSR.
- Shaping the pattern activity using a PSF and a programmable low-pass filter (PLPF).
- Feeding patterns into a 32-bit scan chain around the MIPS core.
- Compacting responses with a MISR-based Output Response Analyzer (ORA).

1.2 Main Components Used

32-bit LFSR The Test Pattern Generator (TPG) is a 32-bit Linear Feedback Shift Register. It produces a maximal-length pseudo-random sequence using the primitive polynomial

$$x^{32} + x^{22} + x^2 + x + 1.$$

Key properties:

- Generates $2^{32} - 1$ non-zero states before repeating.
- Simple hardware: only XOR taps and shift register.
- Seed control allows deterministic but configurable test sequences.

Phase Shifter (PSF) The PSF is a combinational XOR network that takes the LFSR output and re-distributes / decorrelates bits before driving the scan chain input. Typical roles:

- Breaks simple bit-wise correlations from the LFSR.
- Provides better spatial distribution of toggles across scan cells.
- Enables driving multiple scan chains in parallel (if extended).

In this project a single 32-bit chain is used, so the PSF primarily adds decorrelation and slight randomization of bit positions.

Programmable Low-Pass Filter (PLPF) The PLPF is a probabilistic post-processing block that reduces the effective switching factor of the scan-in bit-stream. Conceptually:

- For $n = 1$ (bypass) the toggle probability is $\approx 50\%$.
- For $n = 2$ the expected toggle probability is $\approx 25\%$.
- For $n = 3$ the expected toggle probability is $\approx 12.5\%$.

The PLPF achieves this by combining the current LFSR bit with future bits and a past bit using AND/OR logic controlled by a mode parameter n which is determined by the control signal. Later in the project a dynamic PLPF is introduced, with parameters (α, β, γ) that control which part of the scan chain uses which mode, enabling fine-grained control over power.

Scan Chain The scan chain wraps 32 flip-flops forming a serial path through the MIPS core:

- In scan-in mode (`scan_enable=1`) it shifts in pattern bits from PLPF.
- The 32 bits latched in the chain are interpreted as a MIPS instruction.
- During capture (`scan_enable=0`) the chain latches the ALU result.
- In scan-out mode (`scan_enable=1`) the chain shifts captured responses out towards the MISR.

The scan chain also feeds back its first flip-flop output to the TPG for phase-controlled pattern shaping.

32-bit Single-Cycle MIPS The core under test is a classic single-cycle MIPS-like 32-bit processor:

- Instruction memory, register file, ALU, data memory and control unit.
- Program Counter (PC) and next-PC logic (PC+4 or branch target).
- RISC-V style encoding is used in the implementation (R-type, I-type, etc.).

For LBIST integration, the datapath is modified so that when `scan_enable=0`, the instruction used by the control unit can be driven from the scan chain (subject to opcode validity checks), instead of from instruction memory. The ALU result is also exported as a debug and scan-capture signal.

2 Overall LBIST Architecture

Figure 1 shows the top-level LBIST architecture used in this project.

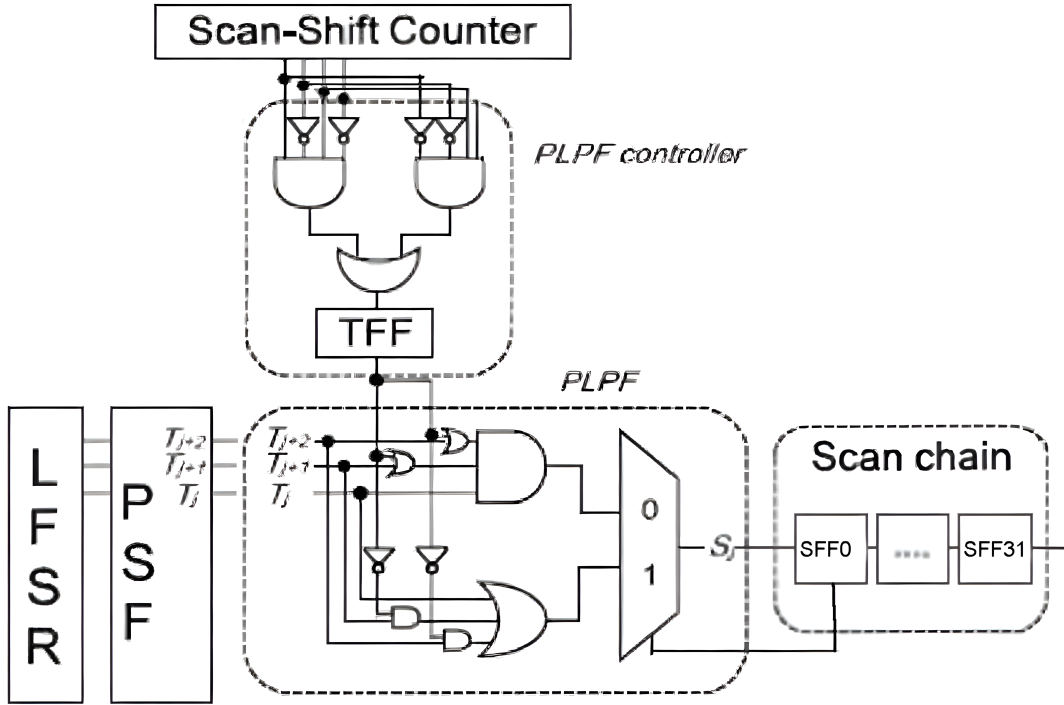


Figure 1: Top-level LBIST architecture around the 32-bit MIPS core.

The main blocks are:

- **LFSR** – 32-bit pseudo-random TPG, seeded via `seed` and `load_seed`.
- **PSF** – XOR-based phase shifter driven by LFSR output.
- **PLPF** – probabilistic filter (static and dynamic) controlled by (α, β, γ) .
- **Scan Chain** – 32-bit serial register connecting to MIPS instruction input and ALU result.
- **MIPS Core** – single-cycle 32-bit datapath and control.

- **MISR** – 32-bit signature analyzer observing `scan_out`.

The testbench applies three main scenarios:

1. Normal MIPS operation from instruction memory (`test_enable=0`, `scan_enable=1`).
2. LBIST with a single scan-in / capture / scan-out sequence.
3. LBIST with multiple patterns, measuring scan-in toggle rate and final MISR signature.

3 Module Descriptions

3.1 32-bit LFSR-Based Test Pattern Generator

The LFSR entity has ports:

- `clk`, `reset`, `enable`;
- `seed`, `load_seed`;
- `lfsr_out` (parallel 32-bit), `serial_out`.

The internal register `lfsr_reg` implements the primitive polynomial:

$$x^{32} + x^{22} + x^2 + x + 1,$$

i.e. feedback bit

$$\text{feedback} = \text{lfsr_reg}[31] \oplus \text{lfsr_reg}[21] \oplus \text{lfsr_reg}[1] \oplus \text{lfsr_reg}[0].$$

On `reset='1'` the LFSR initializes to all ones (non-zero state). When `load_seed='1'` the content is synchronously loaded from `seed`. On each rising edge of `clk` with `enable='1'`, the register shifts and inserts the feedback bit. The output `serial_out` is typically the MSB and drives the PSF/PLPF chain.

3.2 Phase Shifter (PSF)

The PSF is implemented as a simple combinational network that:

- Takes some subset of LFSR bits.
- XORs them in different patterns to form a more randomized scan input.

Although only a single scan chain is used, the PSF still helps decorrelate patterns over time and makes switching more uniform across positions in the 32-bit chain. Its presence is also useful if the architecture is later extended to multiple scan chains.

3.3 Programmable Low-Pass Filter (PLPF)

3.3.1 Static PLPF Modes ($n = 1, 2, 3$)

Initially the PLPF was controlled in the testbench by a discrete mode n :

- $n = 1$: bypass mode, expected toggle $\approx 50\%$.
- $n = 2$: two-bit filter (AND/OR with future bit), expected toggle $\approx 25\%$.
- $n = 3$: three-bit filter, expected toggle $\approx 12.5\%$.

The internal equations for $n = 2$ and $n = 3$ are:

PLPF _{$n=2$} :

$$\begin{aligned} \text{and_n2} &= \text{current_bit} \wedge \text{future_bit1}, \\ \text{or_n2} &= \text{current_bit} \vee \text{future_bit1}, \\ \text{mux_n2} &= \begin{cases} \text{and_n2}, & \text{if past_bit} = 0, \\ \text{or_n2}, & \text{otherwise;} \end{cases} \end{aligned}$$

PLPF _{$n=3$} :

$$\begin{aligned} \text{and_n3} &= \text{current_bit} \wedge \text{future_bit1} \wedge \text{future_bit2}, \\ \text{or_n3} &= \text{current_bit} \vee \text{future_bit1} \vee \text{future_bit2}, \\ \text{mux_n3} &= \begin{cases} \text{and_n3}, & \text{if past_bit} = 0, \\ \text{or_n3}, & \text{otherwise.} \end{cases} \end{aligned}$$

For each n , the testbench measured the toggle rate on the PLPF output (`debug_scan_in`) over several hundred cycles, and the measured values were close to the theoretical expectations.

Table 1: Effect of PLPF input bits on toggle rate(seed = 0xAAAAAAAA).

PLPF input bits	Expected [%]	Simulated [%]
1($n=1$)	50	49
3($n=2$)	16.67	16
5($n=3$)	7.14	7

3.3.2 Dynamic PLPF with (α, β, γ)

To better control power along the scan chain, a dynamic PLPF structure was implemented. For a scan chain of length $L = 32$ and integers α, β, γ satisfying

$$\alpha + \beta + \gamma = 32, \quad \alpha = \gamma,$$

the 32 positions are divided into three regions:

- **Tail:** positions $0 \dots \alpha - 1$ (closest to scan-in).
- **Middle:** positions $\alpha \dots \alpha + \beta - 1$.

- **Head:** positions $\alpha + \beta \dots 31$ (closest to scan-out).

The dynamic selection logic in `plpf_dynamic.vhd` uses the scan-cell index `scan_counter`:

- Tail and head regions use PLPF $n = 3$ (“low toggle” mode).
- The middle region uses $n = 1$ (bypass, “high toggle” mode).

In VHDL:

```
if scan_counter < alpha then
  active_plpf_mode <= "10";  -- n=3
elsif scan_counter < (alpha + beta) then
  active_plpf_mode <= "00";  -- n=1
else
  active_plpf_mode <= "10";  -- n=3
end if;
```

By adjusting (α, β, γ) , we control how large the high-toggle middle region is. The test-bench measures the toggle rate of the TPG/PLPF output `debug_scan_in` and also records the MISR signature. For seed `0xAAAAAAAA` the following results were obtained:

Table 2: Effect of (α, β, γ) on toggle rate and MISR (seed = `0xAAAAAAAA`).

Config	α	β	γ	Expected [%]	Simulated [%]	MISR Signature
A	14	4	14	12–13	11	0x0DEDCBD9
B	10	12	10	23	23	0xAC14C8FE
C	8	16	8	28	27	0x2C35F9AA
D	6	20	6	32	31	0x85FF2B96

Configuration A ($\alpha = 14, \beta = 4, \gamma = 14$) gives the lowest measured toggle rate while preserving sufficient randomness for LBIST and is therefore used as the default setting in the final design.

3.3.3 Seed Dependence

With $(\alpha, \beta, \gamma) = (14, 4, 14)$ fixed, several LFSR seeds were evaluated. As expected, the theoretical toggle rate (12–13%) remains the same, but the actual measured rate and MISR signature vary slightly with the seed:

Table 3: Effect of LFSR seed on toggle rate and MISR ($\alpha=14, \beta=4, \gamma=14$).

Config	Seed (hex)	Expected [%]	Simulated [%]	MISR Signature
S1	AAAAAAAA	12–13	11	0x0DEDCBD9
S2	55555555	12–13	11	0xC2714B31
S3	F0F0F0F0	12–13	10	0x11ADFB30
S4	0F0F0F0F	12–13	15	0x93D6EBD8
S5	12345678	12–13	12	0x166C0827
S6	89ABCDEF	12–13	15	0x5F1DEE98

This demonstrates that:

- PLPF parameters dominate the average toggle rate.
- Different seeds produce different test sequences and thus different MISR signatures, which is desirable for achieving high fault coverage across multiple LBIST sessions.

3.4 Scan Chain Design and Operation

The scan chain is implemented as a 32-bit shift register with ports:

- `clk`, `reset`, `scan_enable`;
- `scan_in`, `scan_out`;
- `mips_instruction` (parallel output);
- `mips_output` (parallel input from ALU result);
- `first_ff_out` (feedback to TPG/PLPF).

In functional terms:

1. **Scan-in phase** (`scan_enable=1`, `test_enable=1`):
on each clock, bits from PLPF enter at `scan_in`, shift through the chain, and after 32 cycles form a 32-bit instruction visible at `mips_instruction`.
2. **Capture phase** (`scan_enable=0`):
the scan chain stops shifting and instead captures the ALU result `mips_output` into its flip-flops on the next clock edge.
3. **Scan-out phase** (`scan_enable=1` again):
the captured response is shifted out serially on `scan_out`, both for observation in the testbench and as input to the MISR.

The output of the first scan flip-flop (`first_ff_out`) is fed back to the LBIST TPG, enabling advanced phase-shifting schemes in the PLPF across successive loads.

3.5 Single-Cycle MIPS Architecture and Modifications

Figure 2 shows the reference 32-bit single-cycle MIPS/RISC-V-like architecture on which the design is based.

The main datapath components are:

- PC register and PC+4 / branch target adder.
- Instruction memory.
- Register file with 32 registers, where register x0 is hard-wired to 0.
- Immediate generator and ALU.
- Data memory and result multiplexor.
- Control unit decoding opcode, funct3, funct7.

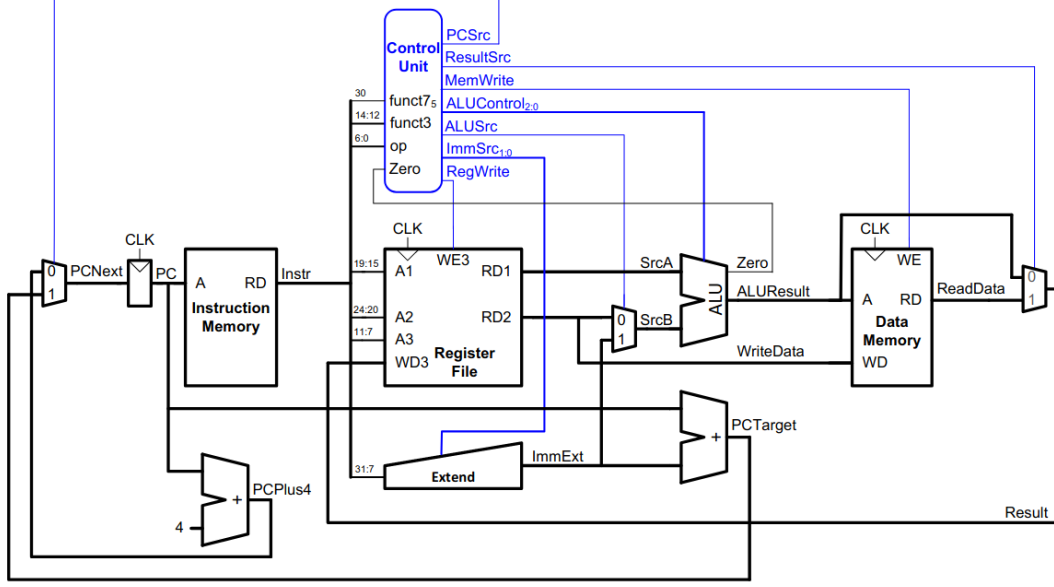


Figure 2: Single-cycle 32-bit MIPS architecture(reference).

Important modifications for LBIST:

1. **Dual instruction source.** In the datapath, the instruction bus `Instr` is driven by:

$$\text{Instr} = \begin{cases} \text{scan_instr_safe}, & \text{scan_enable} = 0, \\ \text{instr_mem_out}, & \text{scan_enable} = 1, \end{cases}$$

where `scan_instr_safe` is the scanned instruction after opcode validity filtering.

2. **Opcode validity and safe instruction mapping.** A function `is_valid_opcode` checks whether the 7-bit opcode belongs to known legal RISC-V groups (R-type, I-type arithmetic, loads, stores, branches, JAL, JALR). For LBIST:

$$\text{scan_instr_safe} = \begin{cases} \text{scan_instruction}, & \text{is_valid_opcode}(\text{scan_instruction}) = \text{true}, \\ 0x002081B3, & \text{otherwise (safe ADD x3,x1,x2)}. \end{cases}$$

This avoids illegal instructions at the core input and ensures that even invalid patterns execute a useful arithmetic operation.

3. **Register file initialization.** The register file initializes several registers to known constants, with `x0` reset to 0 so that the standard NOP `ADDI x0,x0,0` produces ALU result 0, making waveforms easy to interpret.
4. **Debug visibility.** Additional debug signals are exported from the top level:
 - `debug_instruction` (scan chain content),
 - `debug_mips_instr` (instruction actually executed),
 - `debug_mips_output` (ALU result),
 - `debug_scan_in` (PLPF output),
 - `debug_signature` (MISR value).

3.6 MISR and Output Response Analysis

The MISR compresses the `scan_out` stream into a 32-bit signature. It uses the same polynomial as the LFSR:

$$x^{32} + x^{22} + x^2 + x + 1.$$

Internally:

- The MISR register `q` is cleared to all zeros on reset.
- On each rising edge of `clk`, when `enable='1'`, the feedback is computed as

$$\text{feedback} = q_{31} \oplus q_{21} \oplus q_1 \oplus q_0 \oplus \text{data_in},$$

and the new state is

$$q \leftarrow (q_{30}, \dots, q_0, \text{feedback}).$$

At the end of the LBIST session, the testbench reports:

- The final MISR signature.
- An assertion comparing this signature to a golden reference:

$$\text{GOLDEN_SIG} = 0x0DEDCBD9$$

for the baseline configuration $(\alpha, \beta, \gamma) = (14, 4, 14)$ and seed `0xAAAAAAAA`. Any mismatch triggers an error, turning the simulation into a self-checking regression test.

4 Results and Waveforms

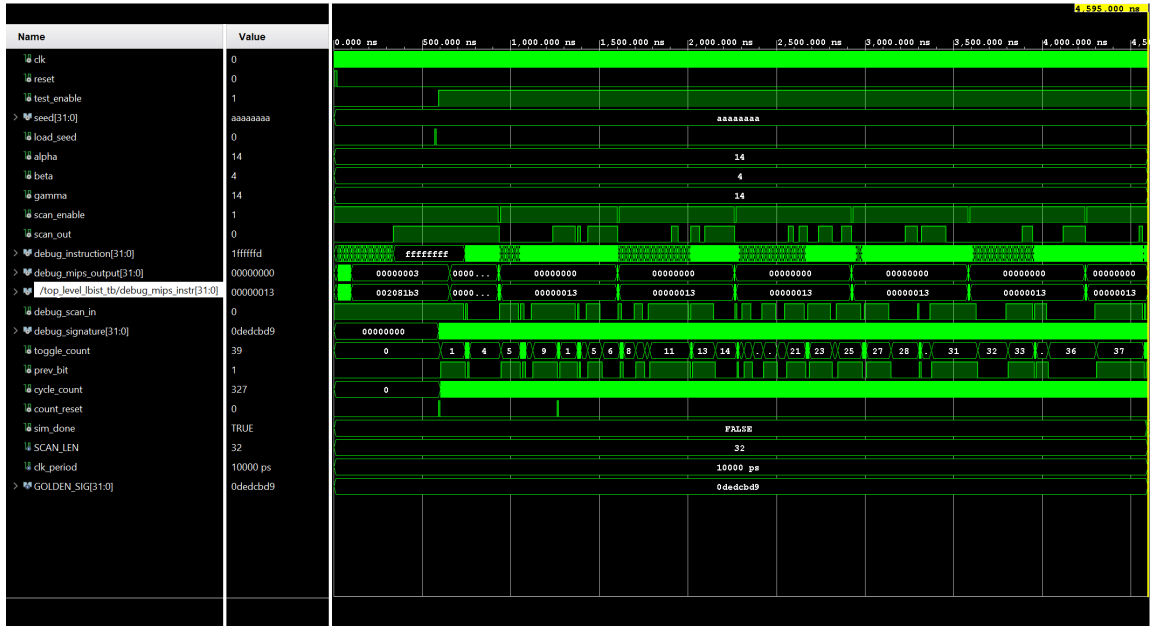


Figure 3: Simulation waveform

5 Conclusion

This project implemented a complete LBIST environment around a 32-bit single-cycle MIPS processor, including:

- A 32-bit LFSR TPG with configurable seed.
- A PSF and programmable low-pass filter (PLPF), with both static and dynamic (α, β, γ) configurations.
- A 32-bit scan chain integrated into the MIPS core for instruction and response access.
- A 32-bit MISR-based Output Response Analyzer with the same polynomial as the LFSR.
- A self-checking VHDL testbench that measures scan-in toggle rate and validates the final MISR signature.

By tuning the (α, β, γ) parameters of the dynamic PLPF, the average toggle rate on the scan-in signal could be reduced from near 50 % down to approximately 11 % for $(14, 4, 14)$, significantly lowering switching power during LBIST. Seed experiments demonstrated that different seeds produce distinct MISR signatures, while the PLPF continues to enforce the desired toggle-rate envelope. The resulting design is a practical demonstration of power-aware LBIST for a small processor core and provides a solid basis for future extensions, such as multi-chain architectures or more advanced pattern-weighting schemes.